

**Engineering, Built Environment and IT
Department of Computer Science**

**Programming Languages
COS 333**

**Examination
17 June 2025**

Examiner: Mr W. S. van Heerden

External Examiner: Mr W. H. K. Bester (Stellenbosch University)

Instructions:

1. Read the questions carefully and answer all questions. Be sure that you understand what the question is asking. If you are unsure, please ask the lecturer who will be moving between the exam venues.
2. This section comprises of **31** questions, and a total of **40** marks.
3. You have **3** hours to complete this test.
4. This test is an *online* assessment: You will be instructed to submit your test when the time (3 hours) expires. You can also submit the test earlier if you are done ahead of time.
5. This test is **closed book** and is subject to the University of Pretoria integrity statement provided below.
 - You are not allowed to consult any sources other than the Practical 2, 3, and 4 specifications, the MIT/GNU Scheme Reference Manual (provided as documentation for Practical 2), and the SWI-Prolog Reference Manual (provided as documentation for Practical 3).
 - For practical implementation questions, your Scheme submission must be working Scheme code that will successfully interpret using version 8.8 of the DrRacket interpreter, while your Prolog submission must be working Prolog code that will successfully interpret using version 9.0.4 of the SWI-Prolog interpreter. You may (and should) use these interpreters to test your submissions. Once you have finished writing each program, make sure each is saved in a text file with the appropriate name (described in the question), and submit it to the correct upload slot. **Be sure to check that you upload the correct program code for each question and be sure to complete the submission (you can check your submission by either looking at the preview of your upload or by downloading your submission). If you fail to attach your program code, there is no way for me to access it, and you will receive a zero mark for the question.**
 - You may use a non-programmable calculator.
 - You may not use any other programmable devices.

Integrity Statement:

The University of Pretoria commits itself to producing academic work of integrity. I affirm that I am aware of and have read the Rules and Policies of the University, more specifically the Disciplinary Procedure and the Tests and Examinations Rules, which prohibit any unethical, dishonest or improper conduct during tests, assignments, examinations and/or any other forms of assessment. I am aware that no student or any other person may assist or attempt to assist another student, or obtain help, or attempt to obtain help from another student or any other person during tests, assessments, assignments, examinations and/or any other forms of assessment.

Question 1

1 Point

Consider the following types of cost that can be associated with a programming language, and **select** the one that is the most relevant to consider in a modern computing context.

- ☐

Cost of compiling programs.
- ☐

Cost of executing programs.
- ☐

Cost of reliability.
- ☐

Cost of the language implementation system.

Question 2

1 Point

Consider the following programming languages, and **select** the one that has the highest level of portability.

- ☐

Fortran I
- ☐

PL/I
- ☐

Swift
- ☐

Java

Question 3

1 Point

Consider the following programming languages, and **select** the one that had the highest overall readability.

- ☐

C++
- ☐

BASIC
- ☐

Ada
- ☐

C#

Question 4

1 Point

Consider the following programming languages, and **select** the one that focused the most on the well-definedness of the original language design (i.e. the first version of the programming language).

☐

Fortran I

☐

ALGOL 68

☐

C

☐

C++

Question 5

5 Points

Write a Scheme function named `getZerosDoublePositives`, which is applied to one parameter. The parameter is a simple numeric list (i.e. a list containing only numeric atoms) called `lst`. The function should yield (not print out) a list containing the values in `lst`, in their original order, where zero values remain unchanged, positive values are doubled, and negative values are omitted.

For example, the function application

```
(getZerosDoublePositives '())
```

should yield an empty list, because the parameter list contains no elements. As another example, the function application

```
(getZerosDoublePositives '(-5))
```

should also yield an empty list, because the parameter list contains only a negative value, which is omitted from the result. As a final example, the function application

```
(getZerosDoublePositives '(-1 4 0 -3 12))
```

should yield the list `(8 0 24)` because the single zero value is included as-is, while 4 and 12 are doubled to produce the values 8 and 24, respectively. Additionally, -1 and -3 are omitted from the result.

Submit your program code in a file named `u99999999.scm` (where 99999999 is your student number) to the assignment submission slot labelled "Examination: Scheme Function", and select the "True" answer below once you have done so. If you do not make a submission, select the "False" answer below. **Be sure to name your file exactly as specified, upload the correct file, and submit your assignment. Failure to do so will result in a zero mark for this question. You can check your submission by either looking at the preview of your upload or by downloading your submission**

Take careful note of the following requirements:

- Your submission must be working Scheme code that will be successfully interpreted by version 8.8 of the DrRacket interpreter (this means, amongst other things, that lowercase letters should be used for all built-in function names). You may (and should) use the DrRacket interpreter to test your submission.
- You may define additional helper functions.
- Your Scheme function may **only** use the following built-in functions in the way they are used in the slides and textbook (**failure to do so will result in a zero mark for this question**):
 - Function construction: `lambda`, `define`
 - Binding: `let` (more complex `let` functions, liked `let*` and a named `let` are not allowed)
 - Arithmetic: `+`, `-`, `*`, `/`, `abs`, `sqrt`, `remainder`, `modulo`, `min`, `max`
 - Boolean values: `#t`, `#f`
 - Equality predicates: `=`, `>`, `<`, `>=`, `<=`, `even?`, `odd?`, `zero?`, `negative?`, `eqv?`, `eq?`
 - Logical predicates: `and`, `or`, `not`
 - List predicates: `list?`, `null?`
 - Conditionals: `if`, `cond`, `else`
 - Quoting: `quote`, `'`
 - List manipulation: `list`, `car`, `cdr`, `cons`
 - Input and output: `display`, `printf`, `newline`, `read`

True

False

Question 6

5 Points

Write a Prolog proposition called `getNegativesNegatePositives(L1, L2)`, where `L1` and `L2` are both simple numeric lists (i.e. lists containing only integers). The `getNegativesNegatePositives` proposition succeeds if `L2` is a list containing the values in `L1` in their original order, where negative values remain unchanged, positive values are negated, and zero values are omitted. For example, the query:

```
?- getNegativesNegatePositives([0], X).
```

should be true with the instantiation `X = []`. This is because there are only zero values in the list `[0]`, which should be omitted from the list `X`. Similarly, the query

```
?- getNegativesNegatePositives([1, 0, -2, 0], X).
```

should be true with the instantiation `X = [-1, -2]`. This is because the positive value of 1 is negated to produce the value -1 and the negative value -2 remains unchanged, while the second and fourth elements are zero values, and are therefore not included in `X`.

Submit your program code in a file named `u99999999.pl` (where 99999999 is your student number) to the assignment submission slot labelled "Examination: Prolog Proposition", and select the "True" answer below once you have done so. If you do not make a submission, select the "False" answer below.

Take careful note of the following requirements:

- Your submission must be working Prolog code that will be successfully interpreted by version 9.0.4 of the SWI-Prolog interpreter. You may (and should) use this SWI-Prolog interpreter to test your submission. For notes on using the interpreter, and re-querying, see the specification for Practical 3.
- You may define additional helper propositions.
- **Note that you may only use the simple constants, variables, list manipulation methods, arithmetic and relational expressions, the `is` operator, cuts, and the built-in `not` proposition discussed in the textbook and slides.** In particular, do not use if-then, if-then-else, and similar constructs. Also, do not use the logical "and" operator, represented using a semicolon (`;`) or pipe (`|`) symbol outside of a list. You may NOT use any more complex predicates provided by the Prolog system itself (including, but not limited to, the built-in `member`, `append`, and `reverse` propositions). In other words, you must write all your own propositions, other than `not`. **Failure to observe this rule will result in a zero mark for this question.**

True

False

Question 7

1 Point

Consider the following program code in a hypothetical programming language:

```
main() {
    var switch
    write("Do you wish to continue?")
    read(switch)

    while(switch) {
        var input
        write("Which action would you like to perform?")
        read(input)
        switch (input) {
            case 1: call action1()
                    break
            case 2: call action2()
                    break
        }
        write("Do you wish to continue?")
        read(switch)
    }
}
```

Assume that `var` introduces a variable definition, `write` prints output to the screen, `read` takes input from the keyboard and stores it in a variable, `while` works like a standard logically controlled loop in the C-based languages, and the `switch` statement works like a multiple-way selection statement in the C-based languages. Also assume that the `action1()`, and `action2()` subprograms are defined correctly and perform arbitrary tasks. **Identify and fill in** the kind of special word that `switch` is in the provided program.

Question 8

1 Point

Consider the following statements about accessing and modifying global variables in PHP, Python, and C, and **select** the one that is the most correct.

- ☐ PHP's approach to accessing and modifying global variables is less writable than both Python and C.
- ☐ PHP's approach to accessing and modifying global variables is less reliable than both Python and C.
- ☐ PHP's approach to accessing and modifying global variables is less readable than both Python and C.
- ☐ PHP's approach to accessing and modifying global variables has a lower cost of execution than both Python and C.

Question 9

1 Point

Fortran supports elemental array operations, which are not supported in most other high-level programming languages. Consider the following reasons, and **select** the one that describes a strong justification for the inclusion of elemental array operations in Fortran.

- ☐ Elemental array operations improve the orthogonality of Fortran.
- ☐ Elemental array operations make sense because their purpose is unambiguous.
- ☐ Elemental array operations make sense for the implementation method that Fortran uses.
- ☐ Elemental array operations make sense in the programming domain for which Fortran was designed.

Question 10

1 Point

Consider the following statements about list comprehensions in Python, and **select** the one that is most correct.

- ☐ List comprehensions improve the readability of list operations in Python.
- ☐ List comprehensions improve the writability of list operations in Python.
- ☐ List comprehensions improve the reliability of list operations in Python.
- ☐ List comprehensions improve the cost of training programmers in Python.

Question 11

1 Point

Consider the following statements about operator precedence and associativity in APL, and **select** the one that is most correct.

- ☐ Operator precedence and associativity in APL is highly orthogonal.
- ☐ Operator precedence and associativity in APL is very complex due to the large number of operators.
- ☐ Operator precedence and associativity in APL introduces a high cost of execution.
- ☐ Operator precedence and associativity in APL is unreliable.

Question 12

1 Point

Consider the following statements about operators in Ruby, and **select** the one that is most correct.

- ☐ Operators in Ruby are implemented in a way that is not very orthogonal.
- ☐ Operators in Ruby are not very writable.
- ☐ Operators in Ruby have a high cost of execution.
- ☐ Operators in Ruby are very reliable.

Question 13

1 Point

Consider the following statements about mixed mode arithmetic expressions in F# and C++, and **select** the one that is most correct.

- ☐ Mixed mode arithmetic expressions in F# are less readable than mixed mode arithmetic expressions in C++.
- ☐ Mixed mode arithmetic expressions in F# are more writable than mixed mode arithmetic expressions in C++.
- ☐ Mixed mode arithmetic expressions in F# have a higher execution cost than mixed mode arithmetic expressions in C++.
- ☐ Mixed mode arithmetic expressions in F# are more reliable than mixed mode arithmetic expressions in C++.

Question 14

1 Point

Consider the following program code in a hypothetical programming language:

```
main() {  
    int input  
    int total = 0  
  
    while ((total <= 100) or ((input = read()) != EOF)) {  
        total = total + input  
    }  
    write(total)  
}
```

The program reads a file containing a list of numbers, and aims to add these values to a total until either the total exceeds 100 or the end of the file is reached, at which point the total will be printed out. Assume that the program uses C-like syntax, `write` prints output to the screen, `read` takes input from the keyboard and returns it, that assignment is an expression (as it is in C), `EOF` indicates the end of file character, and `or` is a logical or. Consider the following statements, and **select** the one that most correctly describes a problem with the above program code.

- ☐ If the `or` operator is short circuit evaluated a side effect will occur, which causes a problem.
- ☐ If the `or` operator is short circuit evaluated a side effect will **not** occur, which causes a problem.
- ☐ If the `or` operator is **not** short circuit evaluated a side effect will occur, which causes a problem.
- ☐ If the `or` operator is **not** short circuit evaluated a side effect will **not** occur, which causes a problem.

Question 15

1 Point

Consider the following statements about conditional targets, and **select** the one that is most correct.

- ☐ Conditional targets reduce the writability of assignments.
- ☐ Conditional targets increase the readability of assignments.
- ☐ Conditional targets increase the cost of execution of assignments.
- ☐ Conditional targets reduce the reliability of assignments.

Question 16

1 Point

Consider the following statements about multiple-target multiple-source assignments in Ruby, and **select** the one that is most correct.

- ☐ Multiple-target multiple-source assignments make Ruby more writable.
- ☐ Multiple-target multiple-source assignments make Ruby more readable.
- ☐ Multiple-target multiple-source assignments make Ruby more reliable.
- ☐ Multiple-target multiple-source assignments decrease the cost of execution of Ruby.

Question 17

1 Point

Consider the following statements about the method that Python and Perl use to disambiguate nested selectors, and **select** the one that is most correct

- ☐ Python's approach to disambiguating nested selectors is less writable than Perl's approach.
- ☐ Python's approach to disambiguating nested selectors is less readable than Perl's approach.
- ☐ Python's approach to disambiguating nested selectors can be less reliable than Perl's approach.
- ☐ Python's approach to disambiguating nested selectors has a higher cost of execution than Perl's approach.

Question 18

1 Point

Consider a situation in which you have two variables, `first` and `second`. You wish to assign different values to `second`, depending on the value of `first`. You decide to do this using a multiple-way selection, where the control expression is `first`. Consider the following programming languages, and **select** the one that allows you to achieve this in the most writable way.

- ☐ C++
- ☐ JavaScript
- ☐ C#
- ☐ Ruby

Question 19

1 Point

Consider the following statements about counter-controlled loops in C++, and **select** the one that is most correct.

- ☐ Counter-controlled loops in C++ are reliable because the loop body always executes a fixed number of times, which is clear from the three control expressions at the start of the loop.
- ☐ Counter-controlled loops in C++ are reliable because it is possible to include the definition of the loop variable in the first control expression at the start of the loop.
- ☐ Counter-controlled loops in C++ are unreliable because it is possible to branch into the loop body.
- ☐ Counter-controlled loops in C++ are unreliable because it is not possible to modify how many times the loop body executes once execution has entered the loop.

Question 20

1 Point

Consider the following statements about user-located loop control mechanisms in Java and C++, and **select** the one that is most correct.

- ☐ Java has more writable user-located loop control mechanisms than C++ does.
- ☐ Java has less writable user-located loop control mechanisms than C++ does.
- ☐ User-located loop control mechanisms are equally writable in Java and C++.
- ☐ Java has less reliable user-located loop control mechanisms than C++ does.

Question 21

1 Point

Consider the following program code in Ruby:

```
def doSomething()  
  num = 3  
  while num > 0  
    yield num  
    num = num - 1  
  end  
end
```

Provide the output of this program code if the `doSomething` subprogram is used with a block that has a single formal parameter, and prints this formal parameter out, followed by a single space. No additional output is printed. Provide just the numeric output, without any punctuation.

Question 22

1 Point

Some programming languages allow default values to be given to a formal parameter when an actual parameter is left out of a subprogram call. Consider the following options, and **select** the one that most correctly describes a mechanism that can be used to allow any actual parameter to be left out of a subprogram call, regardless of the order in which the formal parameters are listed.

- ☐ By using positional parameters.
- ☐ By using keyword parameters.
- ☐ It is impossible to allow any actual parameter to be left out of a subprogram call, regardless of the order in which the formal parameters are listed.
- ☐ Subprograms always allow any actual parameter to be left out of a subprogram call, regardless of the order in which the formal parameters are listed.

Question 23

1 Point

Consider the following statements about subprograms in Ruby, and **select** the one that is most correct.

- ☐ Ruby does not allow a subprogram to have a variable number of parameters.
- ☐ Ruby allows subprograms to have a variable number of parameters, but all the parameters must have the same type because the parameters are stored in an array of a fixed type.
- ☐ Ruby allows subprograms to have a variable number of parameters of any type because the parameters are stored in an array of objects.
- ☐ Ruby allows subprograms to have a variable number of parameters of any type because the parameters are stored in an array, and are all dynamically typed.

Question 24

1 Point

The following C++ program code is not legal:

```
int doSomething(int val) {  
    return val + 2;  
}  
  
float doSomething(int val) {  
    return val + 1.0;  
}
```

Consider the following options, and **select** the one that most correctly describes the underlying reason for this program code being illegal.

- ☐ The fact that C++ does not support overloaded subprograms.
- ☐ The fact that variables of different types cannot be added.
- ☐ The fact that C++ performs a large number of coercions.
- ☐ The fact that C++ performs no coercions.

Question 25

1 Point

Consider the following JavaScript program code:

```
function doSomething(a, b) {  
    return function(y) {return (a + b) - y;}  
}  
  
var thing = doSomething(10, 20);
```

Consider the following options, and **select** the one that most correctly describes what `thing` is.

- ☐ The function
`function(y) {return (a + b) - y;}`
- ☐ The function
`function(y) {return (10 + 20) - y;}`
- ☐ A reference to the `doSomething` function.
- ☐ An invalid function, because a value for `y` cannot be specified.

Question 26

1 Point

Consider the following statements about pass-by-result parameters, and **select** the one that is most correct.

- ☐

Pass-by-result parameters can be passed using a physical copy or an access path.
- ☐

Pass-by-result parameters can be passed using only a physical copy because using an access path would create a functional side effect.
- ☐

Pass-by-result parameters can be passed using only a physical copy because the formal parameter is deallocated once the function terminates.
- ☐

Pass-by-result parameters can be passed using only an access path because a physical copy would take up additional memory and processing time.
- ☐

Pass-by-result parameters can be passed using only an access path because a physical copy would cause the calling function to work with an out of date copy of the formal parameter.

Question 27

1 Point

Consider the following statements about multidimensional arrays as subprogram parameters in C and C#, and **select** the one that is most correct.

- ☐

The approach to passing multidimensional arrays in C is more writable than the approach used in C#.
- ☐

The approach to passing multidimensional arrays in C is more readable than the approach used in C#.
- ☐

The approach to passing multidimensional arrays in C is more reliable than the approach used in C#.
- ☐

The approach to passing multidimensional arrays in C is less orthogonal than the approach used in C#.

Question 28

1 Point

Consider a hypothetical programming language that supports object-oriented programming with multiple inheritance. In this programming language four classes are defined, named A, B, C, and D. Classes B and C both inherit only from class A. Class D inherits from both classes B and C. It is possible for a method name collision to occur **implicitly**. Consider the following implementation details, and **select** only those that together result in an **implicit** name collision (not an explicit name collision). Note that incorrectly selected options will be penalised.

- ☐

Defining a method named `myMethod` in class A.
- ☐

Defining a method named `myMethod` in class B.
- ☐

Defining a method named `myMethod` in class C.
- ☐

Defining a method named `myMethod` in class D.

Question 29

2 Points

You wish to implement a linked list and its nodes. Consider the following programming language constructs, and **select** only the ones that can be used to implement the linked list nodes. Note that incorrectly selected options will be penalised.

- ☐

A struct in C#.
 - ☐

A struct in C++.
 - ☐

A class in C++.
 - ☐

A union in C++.
- Select up to 4 options

Question 30

1 Point

Consider the following statements about Ruby, and **select** the one that is most correct.

- ☐

Ruby has highly efficient primitive types.
- ☐

Ruby has support for a programmer to explicitly allocate and deallocate heap memory for objects.
- ☐

Ruby allows the programmer to choose between dynamic and static message binding.
- ☐

Ruby classes have very unreliable message interfaces.

Question 31

1 Point

Consider the following statements about Java and C++, and **select** the one that is most correct.

- ☐

Java uses objects less exclusively than C++.
- ☐

Java supports some of the benefits of multiple inheritance, while preventing name collisions.
- ☐

Java supports some of the benefits of multiple inheritance, but does not prevent name collisions.
- ☐

Java exclusively supports dynamic message binding, while C++ allows the programmer to choose between dynamic and static message binding.